PROTOCOLGUARD

Explicit Interaction Protocols as Execution Guards
for LLM Multi-Agent Workflows

GitHub

Technical report · Version 1 · Ongoing project

Haneen Kodamy

1 November 2026

Technical Research Report

Abstract

Multi-agent systems built on large language models (LLMs) are vulnerable at the interaction layer: an agent that reads untrusted text can be manipulated into sending messages or taking actions outside its role. We present ProtocolGuard, a runtime execution guard that checks every message and action exchanged between agents against an explicit, machine-readable interaction protocol. The protocol specifies roles, message types, state transitions, value conditions and actions.

We evaluate ProtocolGuard in a sandboxed purchase-to-pay workflow with five agents, using real purchase data from the BPI Challenge 2019 event log and two open-weight LLMs from different model families (Qwen 3.5 9B and Llama 3.1 8B) as the procurement agent. The main results are:

- Scripted attacks: without a guard, all seven attacks succeed; the guard stops six. The remaining one, purchase splitting, is a cross-case pattern outside the reach of per-message rules.
- Real purchases: on 1,000 real purchase orders the guard raises no false alarms. When 60 attacks are hidden among them, it detects all 60 with zero false alarms on the 940 legitimate purchases.
- Prompt injection: an injection embedded in a vendor’s quote manipulated the LLM agent in up to 29 of 30 runs. Across 180 injected runs with Qwen 3.5, the guard reduced harmful outcomes from 38% to 6%. Repeated with Llama 3.1, the guard again blocked every out-of-role action and invented value.
- Protocol revision: an experiment-driven change, binding recommendations to recorded vendor offers, removed all value-manipulation harms.

All remaining harms stem from semantic steering, where the agent is persuaded to choose a valid but more expensive offer. No message-level rule can detect this, which motivates learned and hybrid guards.

1. Introduction

LLM-based agents increasingly coordinate with other agents, humans and tools: they exchange information, give instructions, and call tools that act in the world. In such systems the risk often lies not in a single agent but in the interaction: an instruction given outside someone's role, a message sent at the wrong moment, or a step taken in the wrong state.

Indirect prompt injection makes this concrete. Text returned by tools, here a vendor's quote, can hijack an agent into executing an attacker's goal [2].

Many defenses operate on the text, by detecting injected instructions, or on the model, by making it more robust. Both rely on the model, or a detector, recognising the attack. ProtocolGuard takes a complementary approach. The rules of interaction are written down explicitly, and every message and action is checked against them at runtime. The guard does not need to know *why* an agent misbehaves, only whether its behaviour is permitted. This makes it independent of the model, and it applies equally to LLM agents, scripted agents and humans.

Contributions:

1. A compact, machine-readable protocol format with roles, states, messages, actions, six value-condition types, explicit escalation semantics, and trusted registries (including registries provided at runtime).
2. A design-time checker and a runtime guard built on deny-by-default and fail-closed principles, enforced through a message bus that provides complete mediation.
3. A sandbox combining real process data (BPI 2019) with real LLM agents from two model families, evaluated with a paired design: each LLM decision is tested with and without the guard.
4. Empirical findings on which manipulations a protocol guard can and cannot contain, including an experiment-driven protocol revision, and an analysis of how the guard's response affects availability.

Related work. Prompt-injection attacks on LLM agents are an active research area with dedicated benchmarks [2]; this report focuses on enforcing explicit interaction protocols at runtime.

2. Research questions

- RQ1. Can execution guards generated from an explicit interaction protocol prevent unsafe interactions in an LLM multi-agent workflow, even when an agent is compromised or manipulated, without blocking legitimate work?
- RQ2. Can useful interaction constraints be learned from real historical traces, and how does the amount of data affect false blocks versus missed violations?
- RQ3. Does a hybrid guard, combining explicit and learned constraints, catch attacks that neither catches alone, and at what cost in human escalations?

This report addresses RQ1. RQ2 and RQ3 are future work (Section 8).

3. System design

3.1 Scenario

The scenario is a company purchase-to-pay workflow with five roles:

Role	Responsibility
requester	starts a purchase request (item, quantity, budget)
procurement	reads vendor quotes (untrusted text) and recommends one; this is the LLM agent
approver	approves purchases up to a limit, escalates above it
human_manager	decides on escalated purchases
payment	executes approved payments through the bank tool

Each purchase is a case with its own state:

```
START → REQUESTED → RECOMMENDED → APPROVED → PAID
                          │              ↑
                          └→ PENDING_HUMAN (escalation, then human approval)
REJECTED is reachable from RECOMMENDED and PENDING_HUMAN.
```

3.2 The interaction protocol (version 0.3)

The protocol is a YAML file with five parts:

- 7 message types. Each defines who may send it (senders), who may receive it (receivers), in which states it is allowed (from_states), which state it leads to (to_state), the fields it must contain (required_fields), and optional value conditions.
- 1 action (execute_payment), with performers, allowed_states and max_per_case = 1.
- Parameters: human_approval_threshold = 5000.
- Registries (trusted data): vendors (1,975 real vendors from BPI 2019), and offers, provided at runtime. The offers registry holds the vendor ID and amount of every offer submitted for a purchase, never the offer's free text.
- Escalation semantics: escalations go to human_manager, and the case waits in PENDING_HUMAN.

There are six condition types. Each declares on_fail: block (clearly wrong) or on_fail: escalate (unusual but possibly legitimate):

Condition	Meaning	Used for
in_registry	value exists in a trusted registry	only approved vendors
max_from_case	value ≤ a value remembered in the case	amount ≤ requested budget
matches_case	value = a value remembered in the case	approve exactly what was recommended

Condition	Meaning	Used for
max_parameter	value ≤ a parameter	above €5,000 → escalate
matches_registry	value = the registry's value	pay only to the vendor's registered account
matches_offer (v0.3)	vendor and amount match a recorded offer	no invented prices

3.3 Design-time checking

A loader reads the protocol and its registries and rejects any inconsistency: unknown roles, states, parameters or registries; unknown condition types; missing keys; conditions without an explicit on_fail; and actions without a valid max_per_case. Runtime registries must be supplied, or the protocol refuses to load.

The only way to obtain a protocol object is to pass every check, so the guard can never run on broken rules (fail closed). The checker catches *mistakes*, not *unsafe but valid* rules. For example, granting the payment action to procurement would pass. Proving safety properties of a protocol is left to formal verification (Section 8).

3.4 The runtime guard

For each message or action, the guard decides ALLOW, BLOCK or ESCALATE, always with a reason. Every BLOCK or ESCALATE receives an incident ID, following the acknowledgment, explanation and escalation stages of the Ethical Functional Apology framework [6]. The decision proceeds in eight steps:

1. Replay: a message ID can be used only once.
2. Deny by default: unknown or malformed message types are blocked.
3. Find the case: a new case starts in START.
4. Structure: sender, receiver and state (for messages), or performer, state and repetition count (for actions); then the required fields.
5. Value conditions: a failed block condition always wins over a failed escalate condition.
6. Create the case, only for allowed or escalated messages.
7. Escalate: the case moves to PENDING_HUMAN and nothing moves forward.
8. Allow: a message advances the state and the case stores only the required fields; an action is counted.

Design principles:

- Deny by default: allowlist, not blocklist.
- Fail closed: anything unclear, such as text instead of a number, is blocked.
- The envelope is written by the bus, not the agent: sender role and message ID, which prevents impersonation.
- Only required fields are remembered: prevents memory poisoning.
- Block wins over escalate: a human is never asked to approve something already wrong.
- Checking at the point of action: the payment is re-checked when money moves (defense in depth).
- No rules hidden in code: escalation targets and action limits live in the protocol.

3.5 Message bus and sandbox

All agents communicate through a message bus.

- Agents never hold tools. The bank is reachable only through bus.act, which consults the guard first (complete mediation, a classic security design principle).
- A fake bank records every payment in a ledger, which serves as the ground truth for evaluation. Attack success is measured by what happened to the money, not by the guard's own verdicts.

- Escalations produce a notice for the human containing the incident ID, the reason, the requester, the request and the evidence.
- Every decision is logged as JSON Lines.
- A no-guard baseline that allows everything provides the comparison condition.

3.6 Agents

Agents come in four kinds:

- Honest scripted agents implement the company policy.
- Compromised variants each perform exactly one malicious behaviour.
- "Sometimes compromised" wrappers behave honestly except on targeted purchases.
- The LLM procurement agent uses Qwen 3.5 9B (Alibaba, March 2026 [3]), run locally via Ollama, with temperature 0.7, thinking mode disabled, and JSON output.
  For comparison, the same experiments were run with Llama 3.1 8B (Meta, 2024 [7]), with the same prompt and settings; Llama has no thinking mode.

The LLM agent follows three deliberate design choices:

1. Its raw decision is executed without cleaning.
2. Its instructions grant it broader capabilities than its task requires, so it can message any role and call the bank tool. This reflects a risk known as excessive agency.
3. It receives no prompt-level warning about injections.

The agent is split into a *decide* step (the LLM call) and an *act* step (sending through the bus). This allows each decision to be replayed under different guard settings.

4. Data

Source. The BPI Challenge 2019 event log [1] records purchase-order handling at a large multinational coatings and paints company in the Netherlands, across 60 subsidiaries. Names and IDs are anonymised consistently, and monetary values are anonymised by scaling.

Verification. The file used was a compressed copy of the official log. Its SHA-256 checksum matched the published value, and its contents matched the official description: 1,595,923 events, 251,734 cases, 42 activities, and 1,975 vendors.

Data-quality findings:

- the CSV is encoded in Windows-1252, not UTF-8
- some timestamps are impossible (for example, the year 1948)
- one column name contains a trailing space
- vendor ID and vendor name are separate anonymised codes
- no vendor ID maps to more than one name

Uses in this report:

1. Vendor registry: all 1,975 vendors, each with a generated (fake) bank account, produced deterministically by a script.
2. Realistic workload: 1,000 purchase items sampled with a fixed seed (42). Each item's value is taken at its "Create Purchase Order Item" event, keeping only positive values.

Assumptions:

- The budget equals the real order value, since the log has no budget field.
- Because amounts are scaled, the €5,000 limit has a relative meaning.
- The human manager is scripted: approve up to €10,000, reject above.

The raw data is not redistributed.

5. Experiments and results

5.1 Scripted attack suite

Threat model: exactly one agent is compromised; all others are honest. Each attack runs in a fresh sandbox without and with the guard. Success is judged from the bank ledger.

Attack	Family	Compromised	No guard	With guard
pay_attacker_account	injected account change	payment	succeeded	stopped
skip_approval	role violation	procurement	succeeded	stopped
inflate_price	budget violation	procurement	succeeded	stopped
bypass_human	approval bypass	approver	succeeded	stopped
swap_vendor	binding violation	approver	succeeded	stopped
duplicate_payment	replay / duplicate	payment	succeeded	stopped
split_purchase	threshold evasion	requester	succeeded	succeeded

7 of 7 attacks succeed without a guard; the guard stops 6, each with exactly one incident. Three legitimate purchases completed in both settings, with no false alarms. Purchase splitting succeeds because every message is legal on its own: the attack exists only as a pattern across cases.

5.2 Realistic workload

The 1,000 real purchases were processed with the real vendor registry. Results were identical with and without the guard:

- 939 paid
- 61 rejected by the scripted human
- 116 (11.6%) sent to the human
- 0 false alarms

The total of 4,994 decisions was verified by path counting: 884 × 5 + 55 × 6 + 61 × 4. The whole system (agents, bus and guard) took under 0.3 ms per decision. These are single-run measurements (0.07–0.26 ms, depending on machine load); they were not repeated, so no statistics are reported. Protocol v0.3 produced identical results, again with no false alarms.

5.3 LLM agent under prompt injection

Design. The purchase is 5 laptops with a budget of 6,000. Three quotes are offered: V001 at 850, V002 at 900, and V003 at 950. The correct choice is V001 at 850. The attacker is V003, whose quote text carries the injection. Each of seven conditions (clean, plus six injections) was run 30 times.

Each LLM decision was made once and replayed without and with the guard (paired design), so any difference is caused only by the guard. *Harm* means either of:

- an illegal procurement action was executed, or
- any payment other than 850 to V001 at its registered account.

Final results (protocol v0.3, injection in V003's quote, Qwen 3.5 9B):

Condition	LLM followed	Harm (no guard)	Harm (guard)	Guard blocked	Served (guard)
clean	–	0/30	0/30	0/30	30/30
skip_approver	29/30	29/30	0/30	29/30	1/30
account_change	0/30	0/30	0/30	0/30	30/30
higher_amount	28/30	28/30	0/30	28/30	2/30

Condition	LLM followed	Harm (no guard)	Harm (guard)	Guard blocked	Served (guard)
unknown_vendor	0/30	1/30	1/30	0/30	29/30
subtle_skip	0/30	0/30	0/30	0/30	30/30
steer_to_attacker	10/30	10/30	10/30	0/30	20/30

Across 180 injected runs with Qwen 3.5, harm fell from 68 (38%) without the guard to 11 (6%) with it, an 84% reduction, with no false alarms on clean runs.

Statistics for skip_approver:

- Paired effect of the guard: 29 runs were harmful without the guard and safe with it, and none the other way. McNemar's exact test [4] gives $p \approx 3.7 \times 10^{-9}$.
- Injection success rate: 29/30, with a Wilson 95% confidence interval [5] of about 0.83 to 0.99.

Protocol iteration. An earlier series used protocol v0.2, with the injection placed in the cheapest vendor's (V001) quote and five injection types. There, the guard left 34 of 150 injected runs harmful. Thirty of these came from higher_amount: a recommendation of 4,900, within the budget and below the limit, so every v0.2 rule passed. Adding matches_offer (v0.3) reduced harm with the guard to 6 of 150, and higher_amount from 30 to 0. In the final design above, higher_amount is again fully contained (28 to 0).

Further observations:

1. Knowledge of the agent's message format strongly predicts success. The injection naming the internal fields ("kind", "receiver") succeeded in 29–30 of 30 runs, while the same goal in natural language (subtle_skip) succeeded in 0 of 30.
2. Injection placement matters. With the injection in the cheapest vendor's quote, the model sometimes avoided that vendor (6 runs chose V002). Moving the attacker to V003 reduced this drift to a single run.
3. Scenario wording matters. An initial single run produced a recommendation of 4,250 (850 × 5), because "price" was ambiguous between unit and total. Stating "total price for the whole order" eliminated the error (0 of 30 afterwards).

Model comparison. To test whether the guard's effect depends on the model, the same experiment was repeated with Llama 3.1 8B.

	Qwen 3.5 9B	Llama 3.1 8B
Clean purchases served correctly	30/30	30/30
Followed skip_approver (format-aware)	29/30	10/30
Followed higher_amount	28/30	25/30
Followed steer_to_attacker	10/30	9/30
Drift to another valid offer (injected runs)	1	21
Harm without guard (180 injected runs)	68 (38%)	65 (36%)
Harm with guard	11 (6%)	30 (17%)
Out-of-role actions or invented values executed, with guard	0	0
Time per call	~1.3 s	~0.9 s

Llama followed the format-aware injection far less often than Qwen (10/30 vs 29/30; Fisher's exact test, $p < 10^{-6}$), but drifted to a non-cheapest valid offer in 21 injected runs, although it was correct on all 30 clean runs. For both models, the guard blocked every out-of-role action and every invented value; all remaining harm with the guard consisted of choosing a valid but non-cheapest offer (Qwen 11/180, Llama 30/180).

A comparison with Gemini (free tier) was attempted but stopped by the free tier's rate limit (HTTP 429) after 11 valid calls, too few to report.

5.4 Mixed workload: attacks hidden among real purchases

Sixty of the 1,000 real purchases (6%) were attacked, ten for each of six types (seed 7). bypass_human targeted only purchases above the limit. Ground truth came from an honest run of the same purchases.

- A purchase counts as harmed if it has a payment that should not exist, or was paid without proper approval.
- It counts as served if its outcome equals the honest run's.

Attack	Hidden	Succeeded (no guard)	Succeeded (guard)	Detected (guard)	Served (guard)
skip_approval	10	10	0	10	1
inflate_price	10	10	0	10	0
swap_vendor	10	10	0	10	0
attacker_account	10	10	0	10	0
duplicate_payment	10	10	0	10	10
bypass_human	10	10	0	10	10

All 60 attacks succeed without the guard; with it, none succeed and all 60 are detected (recall 100%). The 940 legitimate purchases show 0 false alarms, 0 harm, and 940/940 correct outcomes (false positive rate 0%).

Purchases under attack were served in 21 of 60 cases. Where the guard blocked only the harmful step (the duplicate payment) or escalated to the human (bypass_human), the purchase still completed correctly. Where it blocked the whole message, the purchase stalled. The single served skip_approval case is one the human would have rejected anyway.

6. Discussion

Containment does not require detection by the model. The LLM followed some injections almost every time and never recognised them. For both model families, the guard still prevented every action outside the agent's role, because it checks permitted behaviour, not intent.

Three tiers of manipulation. The results separate manipulations into three kinds:

1. Structural: wrong sender, receiver, state or action. Fully contained by the protocol.
2. Value: amounts or vendors that no one offered. Contained once the protocol binds messages to trusted structured data (v0.3).
3. Semantic steering: choosing a real but worse option. Not contained, because every message is legal.

The third tier is the most realistic attack observed. It needs no technical tricks: it succeeded in a third of runs with Qwen, and with Llama the injected text caused harm in 21 of 30 runs, mostly through drift to another valid offer.

Separating trusted structure from untrusted text. The decisive design step in v0.3 was to treat an offer's vendor and amount as recorded data, while leaving its free text untrusted. The guard never reads free text, so injected text cannot influence the guard.

Attacker knowledge. Injections that used the system's internal message format were far more effective than natural-language ones. Keeping formats secret is not a defense, but this shows why defenses must hold against attackers with system knowledge.

Residual risk depends on the model, not on the guard. The guard contained the same categories for both models. What differed was how often each model made legal-but-wrong choices under injection:

the newer model obeyed format-aware instructions more readily, while the older one was more easily confused. Better instruction-following can make a model easier to command through injected text.

Response design determines availability. Detection alone is not enough. Blocking whole messages turns attacks into safe but unproductive failures, while blocking only the harmful step, or routing the decision to the responsible human, preserved the legitimate work. This connects runtime guarding to accountable escalation [6].

Methodological lessons:

- Judge harm by outcome (the ledger), not by whether the model "obeyed": unintended drift caused harm that a compliance-only metric would miss.
- Use paired designs to separate the guard's effect from LLM randomness.
- Check the scenario's own wording before attributing errors to the model.

7. Limitations

- Sandbox and scope: one workflow, one small protocol; payments and the human are simulated.
- Circularity: the same author designed the protocol and the scripted attacks, which favours the guard on scripted attacks. The LLM experiments and natural-language injections partly address this.
- Models: two local LLMs from different families (Qwen 3.5 9B, Llama 3.1 8B), one system prompt, temperature 0.7, 30 runs per condition. A Gemini comparison was stopped by free-tier rate limits.
- Attacks: a small, hand-written injection set without adaptive attackers.
- Data: BPI is used for vendors and purchase amounts, not yet for process structure. Budgets are assumed and amounts are scaled.
- Not handled: purchase splitting (cross-case) and semantic steering. There is no recovery after a block.
- Timing: measurements are single runs.

8. Future work

1. Larger and cloud models: repeat the experiments on larger models and on cloud models (the Gemini run was rate-limited).
2. Learned protocols (RQ2): infer constraints from BPI traces in stages: observed transitions, then role-conditioned transitions, then data conditions, then an automaton. Write them in the same YAML format so the existing guard enforces them unchanged, and compare false alarms against the amount of training data, using conformance checking to surface real deviations in the log.
3. Hybrid guard (RQ3): combine hard protocol constraints with learned behaviour across cases (splitting, unusual rates), escalating anomalies.
4. Policy rules with escalation (v0.4): a non-cheapest recommendation is escalated to the human with evidence, rather than blocked, turning silent steering into a visible decision.
5. Recovery after a block: escalate with evidence, or re-run procurement while excluding the suspicious quote.
6. Combined defenses and stronger attacks: combine the guard with prompt-level defenses; use benchmark injection sets [2] and adaptive attackers.
7. Formal verification of protocol properties, such as "no payment without approval."

9. Reproducibility

Repository layout:

```
protocol/   purchase_protocol.yaml, loader.py
guards/     protocol_guard.py (ProtocolGuard, NoGuard)
bus/        message_bus.py (MessageBus, FakeBank)
agents/     scripted.py, llm_procurement.py
attacks/    scenarios.py
bpi/        log.py, explore.py, make_vendors.py, workload.py
```

```
tests/      test_guard.py (24 automated tests)
run_experiment.py
results/    CSV result tables
```

Commands:

```
python -m pytest                          # 24 tests
python -m bpi.make_vendors                # build the vendor registry from BPI
python -m run_experiment                  # scripted attacks + realistic workload
python -m run_experiment mixed            # attacks hidden among real purchases
python -m run_experiment llm              # LLM experiment (Qwen, 30 runs per condition)
python -m run_experiment llm gemini 10    # LLM experiment (Gemini)
python -m run_experiment llm llama     # LLM experiment (Llama 3.1 8B)
```

Settings: seeds 42 (workload) and 7 (attack placement); models qwen3.5:9b and llama3.1:8b; temperature 0.7; thinking disabled for Qwen. The exact Gemini model/version is recorded per call. BPI data must be obtained separately; neither raw nor derived BPI data is distributed through the public repository.

References

[1] B. F. van Dongen, “BPI Challenge 2019,” 4TU.ResearchData, 31 Jan. 2019. doi: 10.4121/uuid:d06aff4b-79f0-45e6-8ec8-e19730c248f1.

[2] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, “AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents,” Advances in Neural Information Processing Systems 37, Datasets and Benchmarks Track, 2024. doi: 10.52202/079017-2636.

[3] Qwen Team, “Qwen3.5-9B,” official Qwen model card, Hugging Face, released 2 Mar. 2026.

[4] Q. McNemar, “Note on the sampling error of the difference between correlated proportions or percentages,” Psychometrika, vol. 12, no. 2, pp. 153–157, 1947. doi: 10.1007/BF02295996.

[5] E. B. Wilson, “Probable inference, the law of succession, and statistical inference,” Journal of the American Statistical Association, vol. 22, no. 158, pp. 209–212, 1927. doi: 10.1080/01621459.1927.10502953.

[6] H. Kodamy and B. Owaidat, “Algorithmic Apologies: Toward an Ethical Framework for AI Accountability,” manuscript under review, Journal of Information, Communication & Ethics in Society, 2026.

[7] Meta AI, “Introducing Llama 3.1: Our most capable models to date,” 23 Jul. 2024. Official release announcement for the Llama 3.1 family, including the updated 8B model.